

Lab 22: K-Nearest Neighbors (KNN) Regression

Aim

To implement the **K-Nearest Neighbors (KNN) algorithm for regression** and predict continuous values.

Algorithm

1. Import the required libraries.
2. Create a dataset with input and continuous output values.
3. Split the dataset into training and testing sets.
4. Create a KNN Regressor with a suitable value of K.
5. Train the model using the training data.
6. Predict the output values for the test data.
7. Calculate and display the Mean Squared Error.

Program

```
# Lab 22: K-Nearest Neighbors (KNN) Regression
```

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

# Create sample dataset
X = np.array([
    [1], [2], [3], [4], [5],
    [6], [7], [8], [9], [10]
])
```

```

y = np.array([
    10, 20, 30, 40, 50,
    60, 70, 80, 90, 100
])

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create KNN Regressor
model = KNeighborsRegressor(n_neighbors=3)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

print("Predictions:", y_pred)
print("Actual Values:", y_test)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

```

Sample Output

Predictions: [43.33333333 53.33333333]

Actual Values: [90 20]

Mean Squared Error: ...

Result

Thus, the **K-Nearest Neighbors (KNN) Regression algorithm** was successfully implemented, and continuous output values were predicted using the nearest training data points.